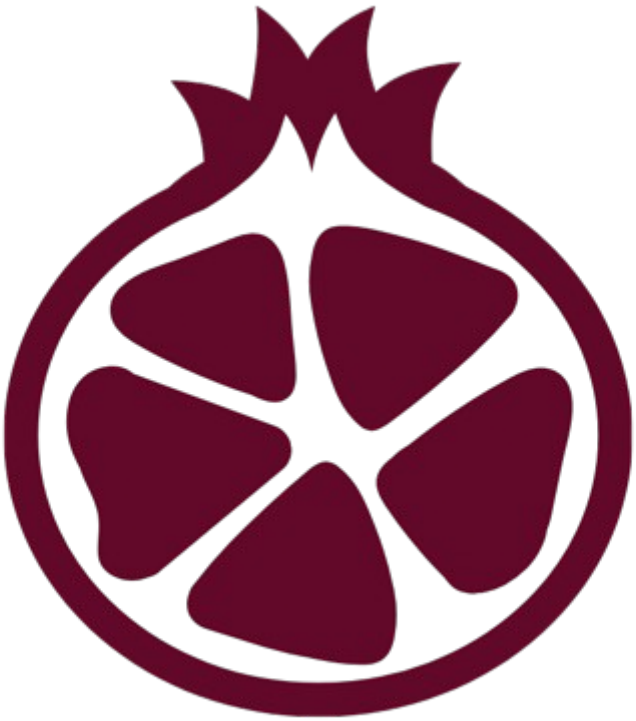

The POM8 Microcontroller Handbook

Table of Contents

Preface . . . 1
1. Development Enviroment Setup . . . 2
  1.1. Vivado Install . . . 2
  1.2. Setting up Devices . . . 3
  1.3. Setting up the Project Files . . . 3
2. The POM8 Architecture . . . 5
  2.1. Program Counter . . . 5
  2.2. Instruction Register & Decode . . . 5
  2.3. Register File . . . 5
  2.4. Arithmetic Logic Unit . . . 5
  2.5. Status Register . . . 6
  2.6. Stack Pointer . . . 6
  2.7. Data Memory Space . . . 6
3. The Instruction Set . . . 8
  3.1. Instruction Set Summary . . . 8
4. Instruction Description . . . 11
  4.1. ADD . . . 11
  4.2. SUB . . . 11
  4.3. AND . . . 12
  4.4. OR . . . 12
  4.5. NOT . . . 13
  4.6. XOR . . . 13
  4.7. LSL . . . 14
  4.8. LSR . . . 14
  4.9. ADDC . . . 15
  4.10. SUBC . . . 15
  4.11. SETC . . . 16
  4.12. CLRC . . . 16
  4.13. SETV . . . 16
  4.14. CLRV . . . 17
  4.15. MOV . . . 17
  4.16. IJMP . . . 17
  4.17. NOP . . . 18
  4.18. CALL . . . 18
  4.19. RET . . . 19
  4.20. JMP . . . 19
  4.21. BRZ . . . 20
  4.22. BRN . . . 20

4.23. BRP . . . 21
4.24. BRC . . . 22
4.25. BRV . . . 22
4.26. HLT . . . 23
4.27. ADDI . . . 23
4.28. SUBI . . . 24
4.29. ANDI . . . 24
4.30. ORI . . . 25
4.31. XORI . . . 26
4.32. LDI . . . 26
4.33. LDA . . . 27
4.34. LDO . . . 27
4.35. STA . . . 28
4.36. PUSH . . . 28
4.37. POP . . . 29
5. Addressing Modes . . . 30
5.1. Direct Addressing . . . 30
5.2. Stack Addressing . . . 30
5.3. Register Addressing . . . 31
5.4. Register Indirect Addressing . . . 31
5.5. Displacement Addressing . . . 31
5.6. Immediate Addressing . . . 32
6. Programming POM8 . . . 33
6.1. POM8 Assembly Syntax . . . 35
6.2. Coding Common Programming Constructs . . . 38
Glossary . . . 42

Preface

This book assumes some basic knowledge in FPGAs and Hardware Description Languages, specifically VHDL. If you are unsure what these terms mean, please take a look at some of the recommended resources below:

- NAND Land FPGA-101
- NAND Land VHDL Tutorials

Chapter 1. Development Enviroment Setup

To begin using POM8, you will need a software suite for the synthesis and analysis of VHDL. You can bring your own preferred software, however this section will focus on the out-of-box experience for a beginner, as such we will discuss the following in this section:

1. **Vivado Install** - Installing the Xilinx Vivado standard edition.
2. **Setting up Devices** - Installing the recommended Digilent devices for Vivado.
3. **Setting up the Project Files** - Importing the POM8 VHDL code into a new Vivado Project.

1.1. Vivado Install

The Vivado installer can be downloaded from here. There are several installers available, however we recommend using the "Self Extract Web Installer" for your operating system.

1. Enter your account details, select "Download and Install now" and click "Next".
2. Select "Vivado" and on the next page select "Vivado ML Standard".
3. You should now be on the installation customisation step. Vivado is a very large software, if you want to minimise disk usage then you can just select the Vivado Design Suite and license key options. This will install a 'blank' version with no devices setup, an issue we will solve in the setting up devices section.

> *This will limit your device options!*
>
> This approach will limit the possible FPGAs you can use with Vivado unless you install them manually later. The following steps will still need to be followed even if all default devices are installed.

4. Follow the remaining steps of the installer and wait...

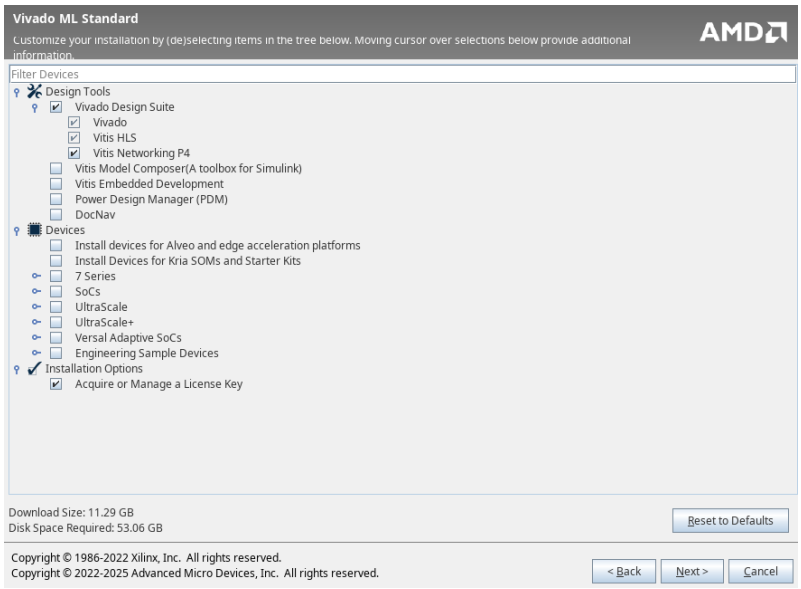

Figure 1. Installation Customisation Step

1.2. Setting up Devices

In this context, device means the FPGA development boards that Vivado can interact with. Setting up a device requires installing the relevant board file, you will need to follow the manufacturers guide on how to get those.

We personally reccomend beginners use the Basys 3 development board by Digilent for two reasons:

- POM8 was hardware tested on the Nexys4 which uses the same Artix-7 chip as the Basys3. As such, we can gaurantee it works out of the box.
- Additionally, it is a capable but relatively inexpensive option when compared to similar development boards.

You can find a guide on how to install the board files from Digilent here.

1.3. Setting up the Project Files

Now that Vivado is completely setup with the relevant devices installed, we can create a new project and import the VHDL code into it. But first, let's download the repository files by cloning it.

```
git clone https://github.com/Zachary-Pearce/POM8.git
```

1. Open up Vivado and click create project under quick start, give it a name, and on the next page select "RTL Project"
2. On the add sources step, click add files and navigate to where you cloned the repository, add all files from the rtl folder, including those in the "packages" and "primitives" sub-folders. The final list should look like the one shown in Figure 2.

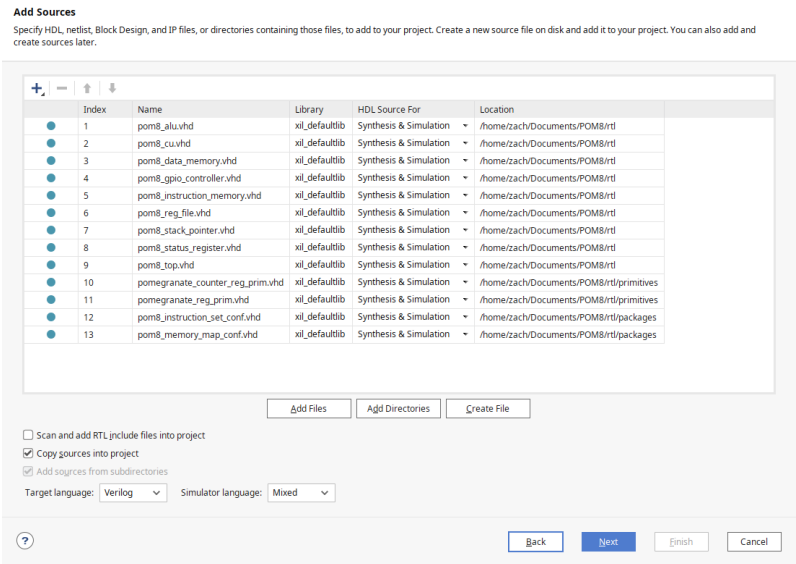

Figure 2. Final List of Added Sources

3. The next step will ask you to add a constraints file, these are provided by the manufacturer for the specific device you are using. For the reccomended Basys3 board, you can find the file and instructions here.
4. Vivado will now ask you to select a default part, this will by default open on the "Parts" tab. However it will be much easier to go to the "Boards" tab and search for the development board you are using there. Just ensure that the board you choose here matches the constraints file you added in the previous step.
5. A preview of the project settings will now be shown, verify that the project settings match what you expect, and finally click "Finish" to create your project. Vivado should now open your project in the "Project Manager" view.
6. Unfortunately, there is one last step before we can get started, we need to import the testbench file which will allow us to run simulations later.
    a. Go to the source window in the top left, right click on "Simulation Sources" and select "Add Sources."
    b. In the same fashion as before, click add files and navigate to the tb folder in the cloned repository. Add the "pom8_top_tb.vhd" file.

Congratulations, you have now setup the POM8 project in Vivado. The remainder of this guide will give a high-level overview of POM8's architecture, explain the syntax of the assembly langauge, and provide some hello world examples.

Chapter 2. The POM8 Architecture

POM8 is an 8-bit microcontroller architecture with a custom Instruction Set Architecture (ISA). It is designed primarily with usability, accessibility, and education in mind. However a reasonable amount of consideration is also given to the performance, this allows the architecture to be an exemplar design. This section will give a very high-level understanding of POM8's architecture as described in Figure 3.

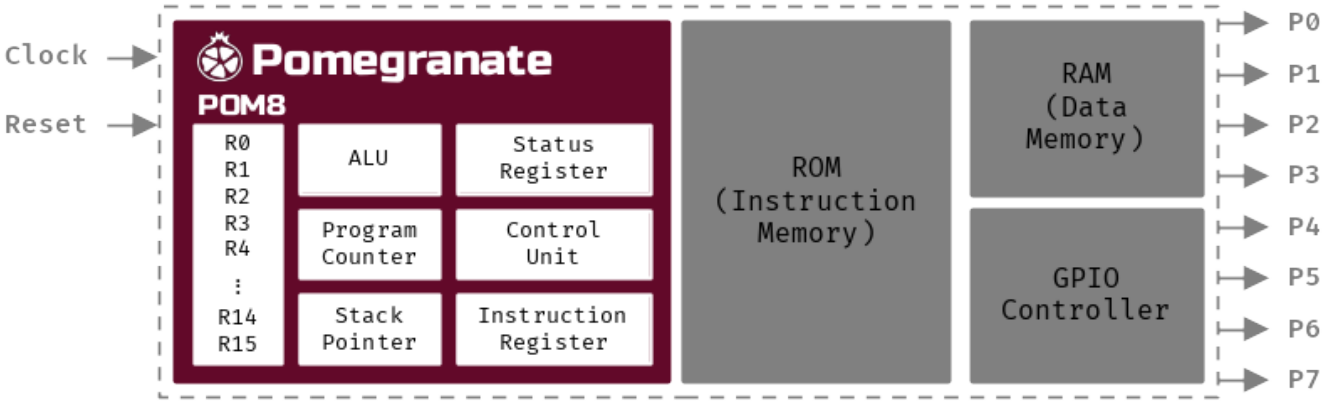

Figure 3. High Level Block Diagram

2.1. Program Counter

The 16-bit Program Counter (PC) is split into a low-byte (PCL) and high-byte (PCH) counter, it holds the address of the current instruction being executed from the instruction memory. When an instruction is fetched from memory, the PC is incremented by increment PCL and then, if a carry is generated, PCH is incremented. When PCH is incremented, we call this crossing a page boundary.

2.2. Instruction Register & Decode

A 3-byte instruction is fetched and written to the Instruction Register (IR), the instruction is then synchronously decoded by the Control Unit (CU) to generate the control signals required for program execution.

2.3. Register File

The register file is an array of 16 general purpose registers each being a word wide. As POM8 is a register-based architecture, the register file holds the result of all operations, it is also used during register indirect addressing.

2.4. Arithmetic Logic Unit

The 8-bit Arithmetic Logic Unit (ALU) performs all arithmetic and logic operations. In addition, the ALU calculates the effective address used in indexed addressing.

2.5. Status Register

The status register is a 5-bit register that is updated after every ALU operation (except incrementing the program counter), it holds the value of the processors various flags:

- Z - Zero flag, set if the result of the last ALU operation was zero.
- N - Negative flag, set if the result of the last ALU operation was a signed negative number.
- P - Positive flag, set if the result of the last ALU operation was a signed positive number.
- C - Carry flag, set if the result of the last ALU operation generated a carry bit.
- V - Overflow flag, set if the addition of two signed integers with the same sign results in the sign bit changing.

The flags are stored in the order of the list above from the MSB to the LSB, as shown in Table 1.

Table 1. Status Register Flags

4	**3**	**2**	**1**	**0**
Z	N	P	C	V

2.6. Stack Pointer

The stack pointer is a 10-bit register that holds the memory address of the top of the stack, each time data is pushed onto the stack, the stack pointer increments its stored address. Conversely, when data is popped off the top of the stack, the stack pointer decrements its stored address.

2.7. Data Memory Space

There are several devices mapped to the data memory space. Table 2 describes the memory map.

Table 2. Memory Mapping

Devices	**Number of Mapped Addresses**	**Starting Address**
RAM	512	0x000
GPIO	4	0x200
RESERVED	508	0x204

2.7.1. RAM

The RAM takes the first 512 words of the data memory space and consists of a single memory module. The RAM is split into several logical partitions as described in Table 3.

Table 3. RAM Partitions

Partition	**Width of Partition**	**Starting Address**	**Description**

Stack	256 words	0x000	Space reserved for the hardware stack implemented in memory.
Free Space	256 words	0x100	Free space, can be used for general program data.

These partitions are not enforced or strict, if you only use 4 words out of the 256 that the stack can hold then you have more free space to play with. However it is important to know the stack exists in memory to avoid overwriting general data.

2.7.2. GPIO

The GPIO (General Purpose Input Output) Controller is an array of 4 registers used to control and output to or read from the eight I/O pins. The GPIO registers are 8-bits wide, each bit corresponding to an I/O pin. The registers are described in Table 4.

Table 4. GPIO Registers

Relative Address	Name	Description
0	in_reg	An 8-bit input buffer.
1	out_reg	An 8-bit output buffer.
2	DDR	An 8-bit Data Direction Register (DDR), if a bit is set, the related I/O pin is an output.
3	RESERVED	No function.

Chapter 3. The Instruction Set

POM8's instruction set is a Reduced Instruction Set Computer (RISC) architecture in which instructions are simple operations. It contains 37 instructions, each three bytes long. First, we will address some of the common terms used in this chapter.

Instructions are grouped based on their function and arguments. These groups are called instruction formats and define where each part of an instruction is located in machine code. Each part will typically be either one of the following:

- Opcode - The operation that the instruction performs, represented by a mnemonic in assembly.
- Operand(s) - The inputs that an instruction uses or manipulates. This may be in the form of an integer, a register, or a memory address.

Programmers will provide these in their code so that the assembler can generate the final machine code. For example, take the following valid line of POM8 assembly:

```
LDA r0, 0x200
```

The opcode is LDA and the operands are register 0 (r0) and a memory address (0x200). This is not the full story; instructions take time to execute. For processors, this time is measured in clock cycles.

Different instructions may require different numbers of clock cycles to complete. Therefore, when writing assembly, choosing instructions with lower cycle counts can improve performance, especially inside loops.

While writing assembly can seem intimidating at first, this is one area where it is simpler than high-level languages. As the execution time and memory usage of each instruction are known, you can make more informed decisions on whether your approach is efficient.

3.1. Instruction Set Summary

An instruction can be given many types of operands:

- Rd - destination register, a register for a value to be written to.
- Rs - source register, a register that provides the first input to an operation.
- Rt - target register, a register that provides the second input to an operation.
- Imm8 - Immediate data, these are 8-bit decimals.
- SAddr - The start address of a subroutine in the instruction memory.
- TAddr - The target address to jump to when performing jumps/branches.
- DAddr - The address of a location in the data memory (RAM).

All instructions require an additional clock cycle when the program counter crosses a page boundary, the instruction set summary shown in Table 5 assumes

the worst case scenario.

Table 5. Instruction Set Summary

Instruction	Description	Operation	#Clock Cycles
ADD Rd, Rs, Rt	Add without carry	Rd = Rs + Rt	4
SUB Rd, Rs, Rt	Subtract without carry	Rd = Rs - Rt	4
AND Rd, Rs, Rt	Logical AND	Rd = Rs AND Rt	4
OR Rd, Rs, Rt	Logical OR	Rd = Rs OR Rt	4
NOT Rd, Rs	Logical NOT	Rd = NOT Rs	4
XOR Rd, Rs, Rt	Logical exclusive OR	Rd = Rs XOR Rt	4
LSL Rd, Rs	Logical shift left	Rd = << Rs	4
LSR Rd, Rs	Logical shift right	Rd = Rs >>	4
ADDC Rd, Rs, Rt	Add with carry	Rd = Rs + Rt + C	4
SUBC Rd, Rs, Rt	Subtract with carry	Rd = Rs - Rt - C	4
SETC	Set the carry status flag	C = 1	4
CLRC	Clear the carry status flag	C = 0	4
SETV	Set the overflow status flag	V = 1	4
CLRV	Clear the overflow status flag	V = 0	4
MOV Rd, Rs	Copy the value of Rs into Rd	Rd = Rs	4
IJMP Rs, Rt	Indirect jump to {Rs, Rt}	PC = {Rs, Rt}	4
NOP	No operation	NA	4
CALL SAddr	Call a subroutine, push value of PC onto the stack	RAM(stack pointer) = PC, PC = SAddr	6
RET	Return from a subroutine, pop value of PC from the stack	PCH = RAM(stack pointer), PCL = RAM(stack pointer)	7
JMP TAddr	Jump to target address	PC = TAddr	4
BRZ TAddr	Jump to target address if Z flag set	PC = TAddr if Z == 1	4
BRN TAddr	Jump to target address if N flag set	PC = TAddr if N == 1	4
BRP TAddr	Jump to target address if P flag set	PC = TAddr if P == 1	4
BRC TAddr	Jump to target address if C flag set	PC = TAddr if C == 1	4
BRV TAddr	Jump to target address if V flag set	PC = TAddr if V == 1	4
HLT	Halt program execution	Halt	4

ADDI Rd, Rs, Imm8	Add immediate without carry	Rd = Rs + Imm8	4
SUBI Rd, Rs, Imm8	Subtract immediate without carry	Rd = Rs - Imm8	4
ANDI Rd, Rs, Imm8	Logical AND with immediate	Rd = Rs AND Imm8	4
ORI Rd, Rs, Imm8	Logical OR with immediate	Rd = Rs OR Imm8	4
XORI Rd, Rs, Imm8	Logical exclusive OR with immediate	Rd = Rs XOR Imm8	4
LDI Rd, Imm8	Load register with immediate	Rd = Imm8	4
LDA Rd, DAddr	Load register with contents of a data memory location	Rd = RAM(DAddr)	5
LDO Rd, Rs, Imm8	Load register indirectly with offset	Rd = RAM(Rs + Imm8)	5
STA Rs, DAddr	Store contents of register in data memory	RAM(DAddr) = Rs	4
PUSH Rs	Push contents of register onto stack	RAM(stack pointer) = Rs	4
POP Rd	Pop contents of register off the stack	Rd = RAM(stack pointer)	5

Chapter 4. Instruction Description

This section will go over each instruction in the set, give a long description of its operation and provide the syntax of the assembly code.

Certain instructions change the value of the status flags, what status flags are changed will be marked using the key shown in Table 6.

Table 6. Flag Status Key

Key	Description
-	Status flag is not changed by this instruction.
x	Status flag is changed by this instruction.

4.1. ADD

Description

Adds the target register (Rt) to the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = Rs + Rt

Z	N	P	C	V
x	x	x	x	x

Syntax

```
ADD Rd, Rs, Rt
```

Example

```
ADD r2, r0, r1  ; add r1 to r0 and store in r2
```

4.2. SUB

Description

Subtracts the target register (Rt) from the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = Rs - Rt

Z	N	P	C	V

x	x	x	x	x

Syntax

```
SUB Rd, Rs, Rt
```

Example

```
SUB r0, r1, r2  ; subtract r2 from r1 and store in r0
```

4.3. AND

Description

Performs a logical AND between the contents of the source register (Rs) and target register (Rt), and places the result in the destination register (Rd).

Operation

Rd = Rs AND Rt

Z	N	P	C	V
x	x	x	-	x

Syntax

```
AND Rd, Rs, Rt
```

Example

```
AND r3, r0, r2  ; AND r0 with r2
```

4.4. OR

Description

Performs a logical OR between the contents of the source register (Rs) and target register (Rt), and places the result in the destination register (Rd).

Operation

Rd = Rs OR Rt

Z	N	P	C	V
x	x	x	-	x

Syntax

```
OR Rd, Rs, Rt
```

Example

```
OR r1, r1, r2   ; OR r1 with r2 and store in r1
```

4.5. NOT

Description

Performs a logical NOT on the contents of the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = NOT Rs

Z	N	P	C	V
x	x	x	-	-

Syntax

```
NOT Rd, Rs
```

Example

```
NOT r1, r0  ; NOT r0 and store in r1
```

4.6. XOR

Description

Performs a logical exclusive OR between the contents of the source register (Rs) and target register (Rt), and places the result in the destination register (Rd).

Operation

Rd = Rs XOR Rt

Z	N	P	C	V
x	x	x	-	x

Syntax

```
XOR Rd, Rs, Rt
```

Example

```
XOR r2, r0, r2  ; XOR r0 with r2 and store in r2
```

4.7. LSL

Description

Performs a logical shift left on the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = << Rs

Z	N	P	C	V
x	x	x	x	-

Syntax

```
LSL Rd, Rs
```

Example

```
LSL r0, r1  ; shift r1 left, store in r0
```

4.8. LSR

Description

Performs a logical shift right on the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = Rs >>

Z	N	P	C	V
x	x	x	x	-

Syntax

```
LSR Rd, Rs
```

Example

```
LSR r0, r1  ; shift r1 right, store in r0
```

4.9. ADDC

Description

Adds the target register (Rt) and carry flag © to the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = Rs + Rt + C

Z	N	P	C	V
x	x	x	x	x

Syntax

```
ADDC Rd, Rs, Rt
```

Example

```
ADDC r0, r1, r2 ; adds r2 and C to r1, store in r0
```

4.10. SUBC

Description

Subtracts the target register (Rt) and carry flag © from the source register (Rs) and places the result in the destination register (Rd).

Operation

Rd = Rs - Rt - C

Z	N	P	C	V
x	x	x	x	x

Syntax

```
SUBC Rd, Rs, Rt
```

Example

```
SUBC r0, r1, r2 ; subtracts r2 and C from r1, store in r0
```

4.11. SETC

Description

Sets the carry flag ©.

Operation

C = 1

Z	N	P	C	V
-	-	-	x	-

Syntax

```
SETC
```

4.12. CLRC

Description

Clears the carry flag ©.

Operation

C = 0

Z	N	P	C	V
-	-	-	x	-

Syntax

```
CLRC
```

4.13. SETV

Description

Sets the overflow flag (V).

Operation

V = 1

Z	N	P	C	V
-	-	-	-	x

Syntax

```
SETV
```

4.14. CLRV

Description

Clears the overflow flag (V).

Operation

V = 0

Z	N	P	C	V
-	-	-	-	x

Syntax

```
CLRV
```

4.15. MOV

Description

Copies the contents of the source register (Rs) into the destination register (Rd).

Operation

Rd = Rs

Z	N	P	C	V
-	-	-	-	-

Syntax

```
MOV Rd, Rs
```

4.16. IJMP

Description

Indirect jump to the instruction address pointed to by the concatenation of the source register (Rs) and target register (Rt).

Operation

PC = {Rs, Rt}

Z	N	P	C	V
-	-	-	-	-

Syntax

```
IJMP Rs, Rt
```

Example

```
; r0 contains 0x00
; r1 contains 0x1B

; indirect jump to address 0x001B
IJMP r0, r1
```

4.17. NOP

Description

No operation.

Operation

No operation.

Z	N	P	C	V
-	-	-	-	-

Syntax

```
NOP
```

4.18. CALL

Description

Call a subroutine by pushing the value of the PC onto the stack and jumping to its start address (SAddr).

Operation

RAM(stack pointer) = PC, PC = SAddr

Z	N	P	C	V
-	-	-	-	-

Syntax

```
CALL SAddr
```

Example

```
; define subroutine using label
sub:    ; some code...

CALL sub

; or use hex addresses directly
CALL 0x0300
```

4.19. RET

Description

Return from a subroutine by popping original PC contents from the stack.

Operation

PCH = RAM(stack pointer), PCL = RAM(stack pointer)

Z	N	P	C	V
-	-	-	-	-

Syntax

```
RET
```

Example

```
; define subroutine using label
sub:    ; some code...
        RET ; return from subroutine

CALL sub    ; call subroutine and save address
; some other code...
```

4.20. JMP

Description

Jump to a target address (TAddr).

Operation

PC = TAddr

Z	N	P	C	V
-	-	-	-	-

Syntax

```
JMP TAddr
```

Example

```
label:  ; label for some code

JMP label   ; jump to label

JMP 0x0030  ; or use hex addresses
```

4.21. BRZ

Description

Jump to a target address (TAddr) if the zero flag (Z) is set.

Operation

PC = TAddr if Z == 1

Z	N	P	C	V
-	-	-	-	-

Syntax

```
BRZ TAddr
```

Example

```
label:  ; label for some code

BRZ label   ; jump to label

BRZ 0x0030  ; or use hex addresses
```

4.22. BRN

Description

Jump to a target address (TAddr) if the negative flag (N) is set.

Operation

PC = TAddr if N == 1

Z	N	P	C	V
-	-	-	-	-

Syntax

```
BRN TAddr
```

Example

```
label:  ; label for some code

BRN label   ; jump to label

BRN 0x0030  ; or use hex addresses
```

4.23. BRP

Description

Jump to a target address (TAddr) if the positive flag (P) is set.

Operation

PC = TAddr if P == 1

Z	N	P	C	V
-	-	-	-	-

Syntax

```
BRP TAddr
```

Example

```
label:  ; label for some code

BRP label   ; jump to label

BRP 0x0030  ; or use hex addresses
```

4.24. BRC

Description

Jump to a target address (TAddr) if the carry flag © is set.

Operation

PC = TAddr if C == 1

Z	N	P	C	V
-	-	-	-	-

Syntax

```
BRC TAddr
```

Example

```
label:  ; label for some code

BRC label   ; jump to label

BRC 0x0030  ; or use hex addresses
```

4.25. BRV

Description

Jump to a target address (TAddr) if the overflow flag (V) is set.

Operation

PC = TAddr if V == 1

Z	N	P	C	V
-	-	-	-	-

Syntax

```
BRV TAddr
```

Example

```
label:  ; label for some code

BRV label   ; jump to label
```

```
BRV 0x0030  ; or use hex addresses
```

4.26. HLT

Description

Halt program execution by entering a halt state, reset POM8 to leave the halt state.

Operation

Enter halt state.

Z	N	P	C	V
-	-	-	-	-

Syntax

```
HLT
```

4.27. ADDI

Description

Add immediate (Imm8) to the source register (Rs) and place the result in the destination register (Rd).

Operation

Rd = Rs + Imm8

- 0 <= Imm8 <= 255
- 0x00 <= Imm8 <= 0xFF
- 0b00000000 <= Imm8 <= 0b11111111

Z	N	P	C	V
x	x	x	x	x

Syntax

```
ADDI Rd, Rs, Imm8
```

Example

```
; immediates can be many data types
ADDI r0, r0, 13         ; decimal
ADDI r0, r0, 0x0D       ; hexadecimal
```

```
ADDI r0, r0, 0b00001101 ; binary
```

4.28. SUBI

Description

Subtract immediate (Imm8) from the source register (Rs) and place the result in the destination register (Rd).

Operation

Rd = Rs - Imm8

- 0 <= Imm8 <= 255
- 0x00 <= Imm8 <= 0xFF
- 0b00000000 <= Imm8 <= 0b11111111

Z	N	P	C	V
x	x	x	x	x

Syntax

```
SUBI Rd, Rs, Imm8
```

Example

```
; immediates can be many data types
SUBI r0, r0, 13         ; decimal
SUBI r0, r0, 0x0D       ; hexadecimal
SUBI r0, r0, 0b00001101 ; binary
```

4.29. ANDI

Description

Perform a logical AND between the source register (Rs) and an immediate (Imm8), and place the result in the destination register (Rd).

Operation

Rd = Rs AND Imm8

- 0 <= Imm8 <= 255
- 0x00 <= Imm8 <= 0xFF
- 0b00000000 <= Imm8 <= 0b11111111

Z	N	P	C	V

x	x	x	-	x

Syntax

```
ANDI Rd, Rs, Imm8
```

Example

```
; immediates can be many data types
ANDI r0, r0, 13         ; decimal
ANDI r0, r0, 0x0D       ; hexadecimal
ANDI r0, r0, 0b00001101 ; binary
```

4.30. ORI

Description

Perform a logical OR between the source register (Rs) and an immediate (Imm8), and place the result in the destination register (Rd).

Operation

Rd = Rs OR Imm8

- 0 <= Imm8 <= 255
- 0x00 <= Imm8 <= 0xFF
- 0b00000000 <= Imm8 <= 0b11111111

Z	N	P	C	V
x	x	x	-	x

Syntax

```
ORI Rd, Rs, Imm8
```

Example

```
; immediates can be many data types
ORI r0, r0, 13         ; decimal
ORI r0, r0, 0x0D       ; hexadecimal
ORI r0, r0, 0b00001101 ; binary
```

4.31. XORI

Description

Perform a logical exclusive OR between the source register (Rs) and an immediate (Imm8), and place the result in the destination register (Rd).

Operation

Rd = Rs XOR Imm8

- 0 <= Imm8 <= 255
- 0x00 <= Imm8 <= 0xFF
- 0b00000000 <= Imm8 <= 0b11111111

Z	N	P	C	V
x	x	x	-	x

Syntax

```
XORI Rd, Rs, Imm8
```

Example

```
; immediates can be many data types
XORI r0, r0, 13         ; decimal
XORI r0, r0, 0x0D       ; hexadecimal
XORI r0, r0, 0b00001101 ; binary
```

4.32. LDI

Description

Place an immediate (Imm8) into the destination register (Rd).

Operation

Rd = Imm8

- 0 <= Imm8 <= 255
- 0x00 <= Imm8 <= 0xFF
- 0b00000000 <= Imm8 <= 0b11111111

Z	N	P	C	V
-	-	-	-	-

Syntax

```
LDI Rd, Imm8
```

Example

```
; immediates can be many data types
LDI r0, 13          ; decimal
LDI r0, 0x0D        ; hexadecimal
LDI r0, 0b00001101  ; binary
```

4.33. LDA

Description

Copy the contents of a data memory address (DAddr) into the destination register (Rd).

Operation

Rd = RAM(DAddr)

Z	N	P	C	V
-	-	-	-	-

Syntax

```
LDA Rd, DAddr
```

Example

```
LDA r0, 0x100
```

4.34. LDO

Description

Fetch the contents of a data memory address provided by the adding an offset (Imm8) to the contents of the source register (Rs), and place it in the destination register (Rd).

Operation

Rd = RAM(Rs + Imm8)

Z	N	P	C	V
-	-	-	-	-

Syntax

```
LDO Rd, Rs, Imm8
```

Example

```
LDO r0, r1, 5
```

4.35. STA

Description

Copy the contents of the source register (Rs) into a data memory address (DAddr).

Operation

RAM(DAddr) = Rs

Z	N	P	C	V
-	-	-	-	-

Syntax

```
STA Rs, DAddr
```

Example

```
STA r0, 0x010
```

4.36. PUSH

Description

Push the contents of the source register (Rs) onto the stack.

Operation

RAM(stack pointer) = Rs

Z	N	P	C	V
-	-	-	-	-

Syntax

```
PUSH Rs
```

Example

```
PUSH r0
```

4.37. POP

Description

Pop a piece of data from the stack and place it in the destination register (Rd).

Operation

Rd = RAM(stack pointer)

Z	N	P	C	V
-	-	-	-	-

Syntax

```
POP Rd
```

Example

```
POP r0
```

Chapter 5. Addressing Modes

In chapter 3 we described operands as the inputs to an instruction, this could be data or an address. Addressing modes are the methods that processors use to identify the location of the operand(s) needed for executing an instruction.

POM8 uses several addressing modes:

- Direct Addressing
- Stack Addressing
- Register Addressing
- Register Indirect Addressing
- Displacement Addressing
- Immediate Addressing

This chapter will discuss these addressing modes in detail and show a datapath to give you an accurate idea of where the operands come from.

5.1. Direct Addressing

In direct addressing, the instruction contains the memory address (A) of the operand, the processor retrieves it from that address.

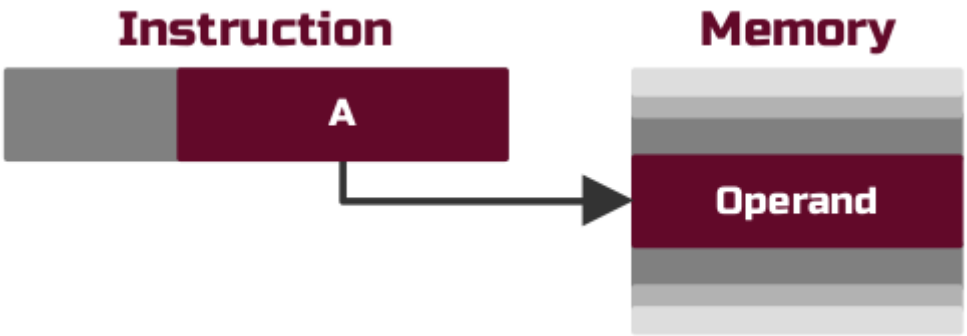

Figure 4. Direct Addressing Datapath

5.2. Stack Addressing

In stack addressing, the operand is implicitly taken from the top of the stack. As such the address is provided by the stack pointer.

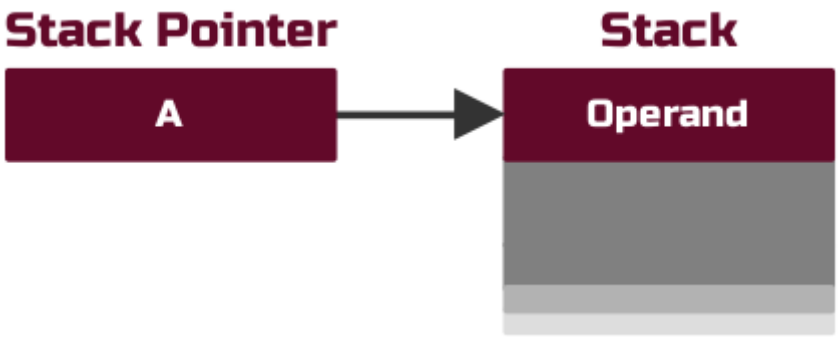

Figure 5. Stack Addressing Datapath

5.3. Register Addressing

In register addressing, the operand is found in a register in the register file at the register address (R) provided in the instruction.

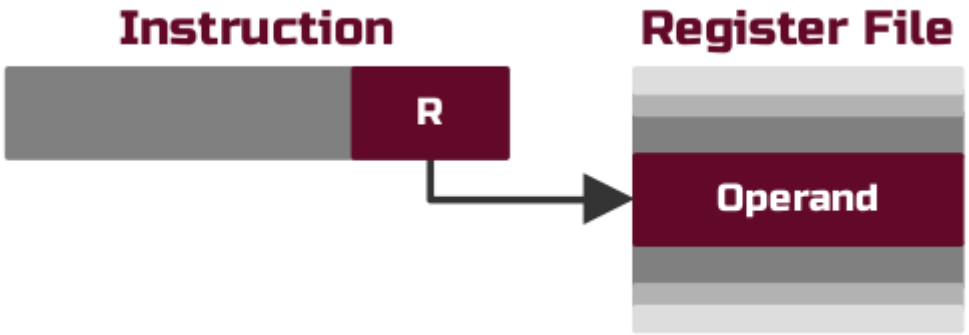

Figure 6. Register Addressing Datapath

5.4. Register Indirect Addressing

In register indirect addressing, the memory address (A) is provided by a register with address (R). The processor then accesses the operand at the provided address.

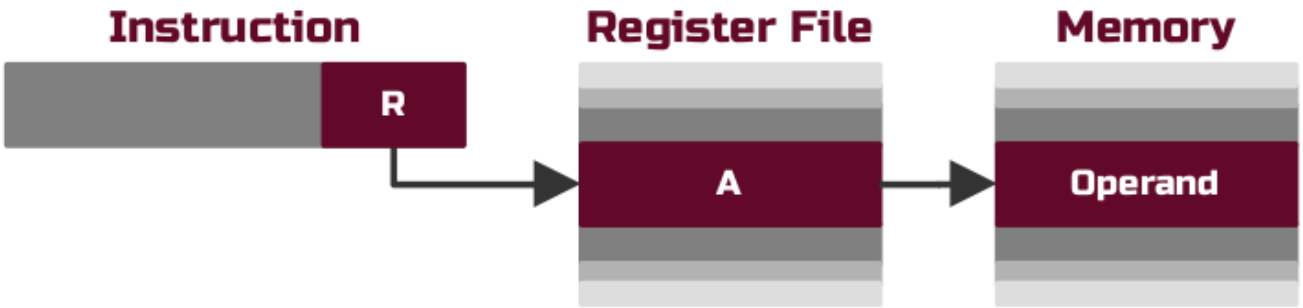

Figure 7. Register Indirect Addressing Datapath

5.5. Displacement Addressing

In displacement addressing, an effective address is calculated by adding a displacement (A) provided in the instruction to the contents of a register at address (R).

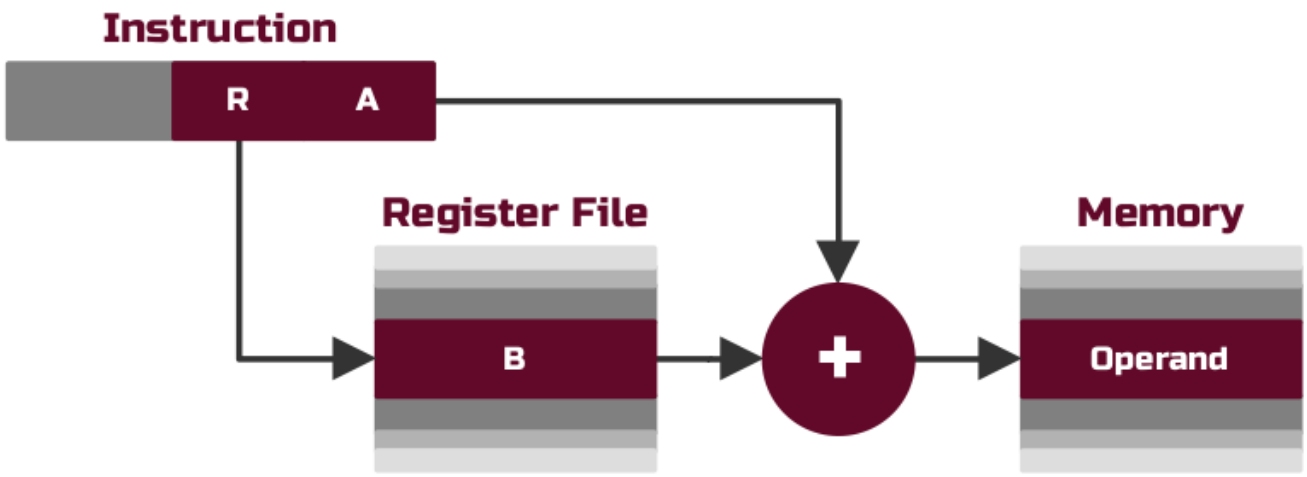

Figure 8. Displacement Addressing Datapath

5.6. Immediate Addressing

In immediate addressing, the operand is provided in the instruction itself.

Figure 9. Immediate Addressing Datapath

Chapter 6. Programming POM8

Programming in assembly is different from high-level languages. Assembly languages use mnemonics to represent instructions while high-level languages use English-like statements.

The process of compilation will convert the high-level code into assembly and then convert the resulting assembly into machine code. Each line of high-level code may translate into multiple lines of assembly.

Another key difference between high-level and assembly code is program flow. In a high-level language, the order in which the code is written is only the order it is defined and not always the order it is executed.

```python
def function():
    print("This is a function")

def main():
    print("This is the main function")
    function()

#execute the main function
main()
```

The above code snippet shows the typical structure of a Python program. In this snippet we have defined a function and a main function. The last line is the first to be executed, calling the main function.

In comparison, assembly code by default is executed in the order in which it is written. Below is an assembly equivalent of the python code above, printing has been abstracted as to focus on the structural differences.

```
        ; execute the main function
        CALL main
        HLT

func:   ; print
        RET

main:   ; print
        CALL func
        RET
```

We know in reality that processors run on a program counter which increments during instruction execution, this counter points to the instruction to be executed from program memory.

In terms of the instruction set, every assembly mnemonic equates to exactly one instruction. POM8 instructions occupy a single address in program memory, this address can be 0x0000 to 0xFFFF. For

simplicity, the first line will be located at address 0x0000, the second at 0x0001, and so on.

If we want to change program flow so instructions are not executed in line order, we must use branches and jumps. Let's annotate the previous assembly with each line's address to demonstrate this.

```
        ; execute the main function
        CALL main   ; 0x0000
        HLT         ; 0x0001

func:   ; print     ; 0x0002
        RET         ; 0x0003

main:   ; print     ; 0x0004
        CALL func   ; 0x0005
        RET         ; 0x0006
```

The processor will always begin at the first address, that being 0x0000, the processor takes the following steps to execute this program.

1. The first line to run is "CALL main", this will call the main function by jumping to its address (0x0004) and saving the address stored in the program counter, 0x0000.
2. The program counter now points to 0x0004, the start of the "main" function. The processor will run the print statement on this line.
3. The program counter increments to 0x0005, the processor will call the "func" function by jumping to its address (0x0002) and saving the address stored in the program counter, 0x0005.
4. The program counter now points to 0x0002, the start of the "func" function. The processor will run the print statement on this line.
5. The program counter increments to 0x0003, the processor will return from the "func" function to where we called it from. Storing the saved address, 0x0005 in the program counter.
6. The program counter increments to 0x0006, the processor will return from the "main" function to where we called it from. Storing the saved address, 0x0000 in the program counter.
7. The program counter increments to 0x0001, the processor will halt program flow and therefore complete the execution of our program.

This may seem foreign to you but this is exactly what the Python program does as well, it is just shown to you through a different lens, through which the low-level details of addresses and program counters is abstracted, hidden from view. This results in the high-level language being more readable, but the way the code is executed is fundamentally the same.

It is clear that the assembly code is different. There is the addition of the HLT instruction to clearly indicate when the program is done as well as the RET instruction that defines when to exit a function. Coding in assembly requires a different thought process to high-level languages however it is by no means a completely fresh start as you can see the overall structure isn't that different.

Assembly also has its own tools to aid readability, we can use labels (such as "main" and "func") to

abstract addresses so long as we understand the rules we need to follow. This chapter aims to help you understand those rules so that you can program POM8 in assembly.

6.1. POM8 Assembly Syntax

Each instruction has an opcode mnemonic and zero or more operands. The order and number of the operands depends on the instruction, Table 5 shows this and the opcode mnemonic. Take the example of an ADD instruction below.

```
ADD r0, r0, r1
```

There must be a space between the instruction mnemonic and the first operand. Each operand is separated by a comma, with optional whitespace. There are multiple data types that can be used for operands:

- Register - Registers are denoted with a prefix, 'r', followed by a digit 0-15 to represent the 16 available registers.
- Hexadecimal - Hexadecimals are denoted with a prefix, "0x", followed by any valid hexadecimal characters (0-9 and A-F).
- Decimal - Decimals are automatically detected when only digits are used, e.g. 1.
- Binary - Binary strings are denoted with a prefix, "0b", followed by any valid binary characters ('0' and '1').

> *Case Sensitivity*
>
> POM8 is a case insensitive language, uppercase and lowercase letters are treated the same. For example, "R0" is the same as "r0" and "ADD" is the same as "add". Labels are an exception to this rule.

The data types that can be used are determined by the operand. Operands also have a range of acceptable values, regardless of the data type. This information is summed up in Table 7 below.

Table 7. Summary of Operand Types and Ranges

Operand	Description	Valid Types	Valid Range
Rd	Destination register.	Register	r0-r15
Rs	Source register.	Register	r0-r15
Rt	Target register.	Register	r0-r15
Imm8	An 8-bit immediate.	1. Hexadecimal 2. Decimal 3. Binary	1. 0x00-0xFF 2. 0-255 3. 0b00000000-0b11111111
SAddr	Subroutine address.	1. Hexadecimal 2. Label	1. 0x0000-0xFFFF 2. N/A

TAddr	Branch/jump target address.	1. Hexadecimal 2. Label	1. 0x0000-0xFFFF 2. N/A
DAddr	Data address.	1. Hexadecimal 2. Decimal 3. Binary	1. 0x000-0x3FF 2. 0-1023 3. 0b0000000000-0b1111111111

6.1.1. Comments

On a line, we can also add comments, any text after a ';' is considered a comment and is ignored by the assembler. As POM8 assembly separates each element on a line with a whitespace, there must be a space before the semicolon, valid syntax is shown below:

```
ADD r0, r0, r1 ; This is a comment
ADD r0, r0, r1 ;This is a comment
```

Below is an example of invalid syntax, in this case, the text after the semicolon will not be considered a comment and may cause errors.

```
ADD r0, r0, r1; This is not a comment
ADD r0, r0, r1;This is not a comment
```

6.1.2. Labels

A label is a named reference to whatever line it is defined on, it can replace a hexadecimal value for jumps, branches, and function calls. A label must:

- Start with a letter.
- Only contain letters and numbers.
- End with a colon.

Here are a few examples of valid labels.

```
label0:
sub16:
add8Inst:
```

Labels cannot be the only element on a line, they must be followed by an instruction, with at least one space between them.

```
add8Inst: ADDI r0, r0, 8
          ; other code...
```

Referencing a label is case sensitive. The name of the label must be written exactly as it was defined. For example...

```
add8Inst: ADDI r0, r0, 8
          ; other code...

          JMP add8Inst ; correct syntax
          JMP add8inst ; incorrect syntax, label does not exist
```

6.1.3. Readability

POM8 assembly is not a fussy language. It will accept any number of spaces, random capital letters, and extremely long label names.

You can write some ugly code, for example...

```
 LdI r0,                0b11110000
                        sTA r0, 0X202 ;setupDDR
start:  lDa r0, 0X200       ;       get     input
            AdDi        r1, r0, 5  ; aDd 5 to inp

LDI             r2,         4
  ThisIsWhenWeShift:      LSL r1, r1      ; ShftReSUlt


      SubI r2,  R2,             1
     BrZ                done
            JMP                 ThisIsWhenWeShift
  done:  StA R1, 0x201   ;        out!!!
        JmP         start
```

The assembler will happily sanitise and compile this, however it is completely unreadable. The language offers a great deal of flexibility to try to ensure you can comfortably code in whatever style you see fit.

However this places more importance on you, as a programmer, to make sure your code is readable and follows a consistent standard. Let's highlight some of the issues with the above code:

- Inconsistent use of capital letters.
- Non-informative comments.
- Inconsistent spacing between elements.
- Use of extremely long label names.

Let's look at a much cleaner version of the code:

```
            LDI r0, 0b11110000
```

```
            STA r0, 0x202   ; store 11110000 in the GPIO DDR

start:      LDA r0, 0x200   ; get input
            ADDI r1, r0, 5  ; add 5 to the input

            LDI r2, 4
shift:      LSL r1, r1      ; shift result to align with output pins
            SUBI r2, r2, 1
            BRZ done
            JMP shift

done:       STA r1, 0x201   ; output result
            JMP start
```

Effort has been put in to make sure that labels are short but informative, capital letters are used consistenly, and comments start in the same place on every line. Additionally, the key parts of the program are visually separated using empty lines.

6.2. Coding Common Programming Constructs

This section will go over creating the common programming constructs, selection, iteration, and functions.

6.2.1. Selection

Selection is what is commonly known as conditional statements, these statements run different code based on some condition(s) being met. For example, the if-else chain in the Python code shown below.

```python
if (var == 0):
    # Some code...
else:
    # Some other code...

# Code that runs after...
```

These lines of Python would convert to the lines of POM8 assembly shown below. The first line of assembly generates status flags depending on the value of r0, if this value is zero then the Z-flag will be set. Then we can check if r0 has a value of zero using the BRZ instruction.

```
        ; use the value of var (r0) to set status flags
        ADDI r0, r0, 0
        ; if var (r0) equals 0, goto "zero" label
        BRZ zero

        ; Some other code...
        ; exit the if statement
```

```
        BRA both

zero:   ; Some code...

both:   ; Code that runs after...
```

The else branch will run if var does not equal zero. In high-level languages, we understand that we exit the if-statement once either:

- All conditions are evaluated.
- Or one condition is true and the code within it has run.

In terms of instruction order, the code marked with the "both" label executes after the code marked by the "zero" label, therefore it can be executed without branching if var equals zero. However, when this is not true, we need to skip over the code for the zero condition. Hence the need to use a BRA instruction to exit the if statement.

6.2.2. Iteration

Iteration is the process of repeating instructions without having to write them multiple times. This often makes code easier to read in high-level languages, but it can also reduce the size of the code in memory.

For Loops

Within high-level languages, the most common types of loop are the for loop and the while loop. First we are going to look at the for loop, Python example shown below.

```python
for i in range(5):
        print(f"The index is: {i}")
```

This loop will print the value of the iterator, i, which starts at zero. For each iteration:

1. We check if i is within the defined range, the stop value of five being non-inclusive. The range is therefore 0, 1, 2, 3, 4. Meaning that at the start of each iteration, we check if i is more than or equal to five.
2. At the end of each iteration, i is incremented.

Let's see what this code looks like in POM8 assembly. Once again with printing abstracted.

```
        ; setup the loop
        LDI r0, 0       ; initialise i

loop:   ; print i
        ADDI r0, r0, 1  ; increment i
        SUBI r1, r0, 5
        BRN loop        ; if i is less than 5, loop
```

```
8
9         HLT             ; if i was more than 5, halt
```

Line 2 initialises i at zero, this step is not technically required but is done for demonstration purposes. During the loop, we:

1. print i
2. Check if i is within range by subtracting five.
3. If the result of subtraction is negative with the N-flag being set then we know that i is less than five and we branch to the loop label on line four to start the next iteration.

Once i is more than or equal to five, we exit the loop and halt program execution on line nine. The order of events is slightly different here than in Python, in Python we check the condition at the start of the loop then increment at the end of it, but in assembly we check the condition after incrementing at the end of the loop. There is no real difference in how these approaches behave, however the latter ends up reading much nicer in assembly.

While Loops

While loops are condition controlled, they will loop while a certain condition condition is met.

```python
i = 5
while i == 5:
        print("Increment i? (Y or N)")
        x = input()
        if x == "Y":
                i = i + 1
```

The above loops while i is equal to 5, each iteration it asks the user if they want to increment i, if they answer is "Y" then i is incremented. The condition for a while loop is evaluated at the start of each iteration, if the condition is true then the code in the loop is exectued, otherwise we exit the loop.

Let's write this code in POM8 assembly. We will take our input from one of our GPIO pins, we will setup the pins using the Data Direction Register (DDR) and read values from the input register (in_reg). We can imagine that a button is connected to pin 1, if this button was pressed then we want to increment i.

```
1         LDI r0, 0b00000000
2         STA r0, 0x202   ; initialise DDR
3         LDI r0, 5       ; initialise i as 5
4
5         JMP loop        ; jump to loop condition first
6
7         ; loop contents
8 cloop:  LDA r1, 0x200   ; get input from GPIO pins
9         ANDI r1, r1, 0b00000001 ; check for input on pin 1 (P0)
```

```
10          BRZ loop        ; if zero then no input, skip to loop condition
11          ADDI r0, r0, 1  ; otherwise increment i
12
13          ; loop condition
14 loop:    SUBI r1, r0, 5  ; set status flags for i == 5
15          BRZ cloop       ; branch to loop content if i == 5
16
17          HLT             ; loop condition isn't met, halt
```

The first 3 lines initialise the Data Direction Register (DDR) and i, in this case we give i its starting value of five and set all GPIO pins as inputs.

The loop itself is a bit more complicated, let's focus on the structure for now. The while loop has 2 parts, the condition and the contents. When entering the loop, the first thing we do is jump to the loop condition, then if the condition is true we branch to the loop contents. Otherwise we exit the loop and halt the program.

You may be wondering why we don't just have the loop condition on lines 5 and 6 before the loop contents. On paper it may seem more efficient as we don't have to jump to it when we enter the loop, but let's see what that looks like.

```
1           LDI r0, 0b00000000
2           STA r0, 0x202   ; initialise DDR
3           LDI r0, 5       ; initialise i as 5
4
5           ; loop condition
6 loop:     SUBI r1, r0, 5  ; set status flags for i == 5
7           BRZ cloop       ; branch to loop content if i == 5
8           JMP done        ; loop condition not met, exit loop
9
10          ; loop contents
11 cloop:   LDA r1, 0x200   ; get input from GPIO pins
12          ANDI r1, r1, 0b00000001 ; check for input on pin 1 (P0)
13          BRZ loop        ; if zero then no input, skip to loop condition
14          ADDI r0, r0, 1  ; otherwise increment i
15          JMP loop        ; jump to loop condition
16
17 done:    HLT
```

This structure has added an additional jump per iteration as we need to continually jump to the condition after each iteration. It also added another jump for exiting the loop.

This massively decreases the performance compared to the previous structure. No jump is needed to exit the loop as the code following the loop is directly after the condition check, as such it will run if the condition is false and no branching occurs.

Additionally, rather than jumping back to the condition after each iteration, we jump to the condition once when we enter the loop.

Glossary

FPGA

Field Programmable Gate Array, a programmable integrated circuit that consists of an array of programmable logic blocks which can be configured and interconnected to perform various digital functions.

Machine Code

Binary instructions, that being sequences of 1s and 0s, that a processor can directly execute.

Mnemonic

In assembly languages, a mnemonic is a symbolic name used to represent a machine instruction. Each mnemonic typically responds to a specific operation the processor can perform, such as moving data.

Opcode

The operation that the instruction performs, represented by a mnemonic in assembly languages.

Operand

The inputs that an instruction uses or manipulates. This may be in the form of an integer, a register, or a memory address.

Word

A processors natural unit of data, it is the amount of data that a processor can process in one go. The word width is given in bits, where an 8-bit processor would have a word width of eight bits.